

Reviewer Pack — Minimal Volume of Intersection Body and ACC⁰ Parity Circuit ...

Entry ce070d09a267 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 19:54:37 UTC

Minimal Volume of Intersection Body and ACC⁰ Parity Circuit Complexity

Entry ID: ce070d09a267 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 19:54:37 UTC

1. Conjecture

Field A (mathematical branch): Convex Geometry **Field B** (complexity object): ACC⁰ Parity Circuit Complexity

Statement:

{‘sentence1’: ‘For every explicit function $f: \{0, \dots, N-1\} \rightarrow \{0, 1\}$, the minimal volume of the intersection body of its associated convex polytope is polynomially related to the size of the smallest ACC⁰ parity circuit computing f .’, ‘sentence2’: ‘More formally, there exists a constant $c > 0$ such that for all N and all explicit functions f , the following inequality holds: $\text{Vol}(\text{IntersectionBody}(f)) = \Theta(c * \text{ACC}^0(f))$.’, ‘sentence3’: ‘Here, Vol denotes the volume of the intersection body, and $\text{ACC}^0(f)$ is the size of the smallest ACC⁰ parity circuit computing f .’}

Rationale (proposer’s reasoning):

{‘sentence1’: ‘Convex geometry has been used to study communication complexity, but its application to studying circuit complexity, particularly for parity functions, is rare.’, ‘sentence2’: ‘The intersection body provides a geometric representation of the function that could expose underlying structure and potentially lead to lower bounds.’, ‘sentence3’: ‘This conjecture aims to bridge convex geometry with ACC⁰ parity circuits, which is a promising direction due to the scarcity of natural properties for this complexity class.’}

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e0479bd3cc8d9db2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all tested functions f with $n \leq 40$, the ratio of the volume of the intersection body ($\text{Vol}(\text{IntersectionBody}(f))$) to the size of the smallest ACC⁰ parity circuit ($\text{ACC}^0(f)$) is within a constant factor c , where $\text{support_fraction} = |c - \text{Vol}(\text{IntersectionBody}(f))/\text{ACC}^0(f)| \leq 0.1$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'convex geometry' AND 'ACC0 parity circuit complexity' AND 'intersection body volume' - 'polynomial relationship' AND 'volume of intersection body' AND 'ACC0 parity circuits' - 'constant c' AND 'Vol(IntersectionBody)' AND 'ACC0(f) size'

Top relevant hits considered: - [<http://arxiv.org/abs/1601.05252v2>] Complex hyperbolic volume and intersection of boundary divisors in moduli spaces of genus zero curves - [<http://arxiv.org/abs/1607.06161v3>] Correspondences between convex geometry and complex geometry - [<http://arxiv.org/abs/1305.6023v3>] A Robust Version of Convex Integral Functionals - [<http://arxiv.org/abs/math/9807128v2>] Stratified simplices and intersection homology - [<http://arxiv.org/abs/1401.4438v1>] Integral closure of rings of integer-valued polynomials on algebras - [<http://arxiv.org/abs/math/0307153v1>] Intersection Alexander polynomials - [<http://arxiv.org/abs/1909.13382v3>] A classification of finite simple amenable $\mathbb{Z}$ -stable C -algebras, II, $-C$ -algebras with rational generalized tracial rank - [<http://arxiv.org/abs/1212.4082v7>] Irrationality of the Zeta Constants

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.5s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_convex_polytope(f):
        # Placeholder function to simulate convex polytope computation
        # This is a dummy implementation and should be replaced with actual logic
        return random.random()

    def compute_intersection_body(polytope):
        # Placeholder function to simulate intersection body computation
        # This is a dummy implementation and should be replaced with actual logic
        return polytope ** 2

    def compute_acc0_circuit(f):
```

```

# Placeholder function to simulate ACC0 parity circuit computation
# This is a dummy implementation and should be replaced with actual logic
return random.randint(1, 10)

n = random.choice([5, 10, 15, 20, 30, 40])
f = generate_function(n)
polytope = compute_convex_polytope(f)
intersection_body = compute_intersection_body(polytope)
acc0_circuit_size = compute_acc0_circuit(f)

if acc0_circuit_size == 0:
    return {
        "metric_name": "Vol(IntersectionBody(f)) / ACC0(f)",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "acc0_circuit_size_zero"
    }

ratio = Fraction(intersection_body, acc0_circuit_size)
support_fraction = abs(1 - ratio)

return {
    "metric_name": "Vol(IntersectionBody(f)) / ACC0(f)",
    "metric_value": float(ratio),
    "instances_tested": 1,
    "conjecture_holds": support_fraction <= 0.1,
    "counterexample": "" if support_fraction <= 0.1 else f"support_fraction={support_fraction}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000003) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean = sum(r["metric_value"] for r in results) / len(results)
        std_dev = (sum((r["metric_value"] - mean) ** 2 for r in results) / len(results)) ** 0.5
        support_fraction = sum(1 for r in results if abs(1 - r["metric_value"]) <= 0.1) / len(results)
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"support_fraction_exceeds_0.1\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cbd23b12.py", line 71, in <module>
    result = run_trial(seed)
             ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cbd23b12.py", line 54, in run_trial
    ratio = Fraction(intersection_body, acc0_circuit_size)
             ~~~~~
  File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be ")
TypeError: both arguments should be Rational instances
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture's support conditions. | next: Investigate the cause of the crash and attempt to run the test again.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15660
2	preregistration	ollama_remote	glm4:latest	0	0	9405
3	novelty	ollama_remote	glm4:latest	0	0	8431
4	novelty	ollama_remote	glm4:latest	0	0	9846
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14921
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8190
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8594
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9558
9	judge	ollama_remote	glm4:latest	0	0	15103

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 99709 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/ce070d09a267.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ce070d09a267.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ce070d09a267.tar.gz` (if generated)